

Intercepting Microsoft 365 Copilot traffic: one real blocker left, and what closing it costs

Written 2026-09-21, as a follow-up to [copilot-desktop-apps-coverage-gap.html](#)'s disambiguation of the word "Copilot." That doc ruled Microsoft 365 Copilot **out of scope permanently** for a simple reason: it's a different company's product, talking to a different backend, with no shared infrastructure to intercept via anything built for GitHub Copilot. This doc investigates that question properly on its own terms: **could a local listener the client is redirected to, relaying to the real backend, work against Microsoft 365 Copilot specifically?**

Short answer, updated 2026-09-22: one of the two original blockers is now closed, confirmed against real code, not guessed. This doc originally found two problems — no redirect lever (Reason 1), and a WebSocket protocol this codebase's relay infrastructure wasn't shaped for (Reason 2). Reason 2 is now confirmed **closed**: Optimus already has a native, production WSS relay with content inspection and enforcement built in (see the update under Reason 2 below). Reason 1 remains open, with two ranked, buildable paths already given their own design docs. See **Net assessment** for where that leaves this.

What Microsoft 365 Copilot actually is

Microsoft's own public description: an AI assistant integrated into Word, Excel, PowerPoint, Outlook, Teams, Loop, and Whiteboard. A user's prompt is grounded in their actual organizational data via **Microsoft Graph**, then sent to a large language model on **Azure OpenAI Service**, with the response grounded back into the document/app context. Confirmed distinct from GitHub Copilot on Microsoft's own page: *"a completely separate product from GitHub Copilot."*

Reason 1: there is no redirect lever — the endpoint is fixed, not configurable

Per Microsoft's own admin documentation ([App and network requirements for Microsoft Copilot admins](#)):

- Copilot's enterprise experiences connect to a fixed set of domains: `copilot.cloud.microsoft`, more broadly `*.cloud.microsoft`, and `*.office.com`. These are the same for every tenant — not resolved from any per-tenant or per-device setting.
- Microsoft's explicit guidance to admins is to allow the **entire** `*.cloud.microsoft` domain, stating directly: *"Microsoft doesn't support allowing partial or only selected Microsoft 365*

*application URLs within the *.cloud.microsoft domain."* That's a statement about network policy, but it doubles as confirmation there's no narrower, single addressable Copilot-specific endpoint an admin (or a config write) could isolate and redirect — the whole wildcard domain is the unit Microsoft expects you to treat atomically.

- No config file, registry key, environment variable, or admin policy surfaced in this documentation that repoints where any of this traffic goes.

Consequence: there's no way to write a single config value and have the client connect somewhere else instead — nothing here determines its own destination from anything write-accessible. This closes the question via the app-config route — but a config key isn't the only way to redirect a connection, so the network-layer alternative below was worked through too, before concluding it doesn't actually get around this either.

Four network-layer alternatives to Reason 1, ranked (internal design discussion, 2026-09-22)

Considered as alternatives once the app-config route above closed: instead of asking the *application* to redirect itself, redirect at the *network* layer, the same way the resident daemon already can for other agents. A design discussion on 2026-09-22 walked through four such options and ranked them — worth recording in that order, since two of the four share a disqualifying property and the other two don't.

1 & 2 — system-level proxy config, and pure network/route-level redirection. Two mechanically different levers (OS proxy settings vs. IP routing/egress tables) that share the same disqualifying property: **neither has any notion of which agent is calling.** A system-level proxy rule or a route scoped to *.cloud.microsoft catches that traffic for every process on the box — the browser, other Office apps, anything — not just the one agent worth instrumenting. Functionally this is the same blast-radius problem as the DNS-wildcard approach below, arrived at independently: no per-process context means no way to scope the redirect narrower than the whole domain.

3 — process-level proxy settings. Two sub-variants:

- **3A — a base-URL key in the app's own settings file.** Ruled out for the same reason as the rest of Reason 1: no such key exists for this app.
- **3B — per-process proxy environment/settings, applied only to the target process.** The option to check first, since if it applies it closes Reason 1 for zero new engineering. **Full design and checkpoint findings:** option-3b-per-process-proxy-design.md — short version: the classic Office networking stack (WinINET/WinHTTP) is confirmed to have no per-process lever, but the Copilot pane is WebView2-hosted, and WebView2's own WEBVIEW2_ADDITIONAL_BROWSER_ARGUMENTS is a real, confirmed, potentially process-scoped proxy override — genuinely open pending two concrete live checks documented there.

4 — process-aware kernel-level redirect (WFP on Windows, Network Extension on macOS).

The general-purpose fallback if 3B doesn't apply, built on infrastructure this codebase already has (`network_egress` 's existing process↔connection correlation). **Full design and checkpoint**

findings: `option-4-process-aware-redirect-design.md` — short version: Windows'

`FWPM_LAYER_ALE_CONNECT_REDIRECT_V4` is a real, documented sibling to the observe layer already in use; macOS needs a second, separate system extension (`NETransparentProxyProvider` , distinct from the existing `NEFilterDataProvider`), with a real historical WebSocket-breaking compatibility bug between the two types flagged and resolved (fixed in macOS 11.2 — not a blocker on a 2026 fleet, but worth a pinned regression test).

Sequencing decided in that discussion: check 3B first, since if it applies it's free. If it doesn't, 4 is the fallback — "we can definitely do it there" — accepted as more general, cross-platform, and requiring real time investment and product buy-in, but not blocked on anything Microsoft controls.

The DNS-wildcard variant, for completeness

A fifth way to get the same "no per-process context" redirect as options 1 and 2 above, using DNS instead of proxy config or routes: since a hosts file can't represent `*.cloud.microsoft` / `*.office.com` as wildcards (Microsoft won't publish the real underlying FQDNs — the same "hyperscale and dynamic" reasoning quoted above), a **local DNS resolver that performs real wildcard matching** would be needed instead — mechanically doable, but subject to the exact same blast-radius problem as 1 and 2: it catches SharePoint, OneDrive, Teams, and Outlook along with Copilot, since none of them are distinguished at the DNS layer. Whichever network-layer redirect is used (1, 2, or this one), the client still expects a valid certificate for the real hostname — closing that gap needs the same CA-trust mechanism already used elsewhere in this codebase: mint a leaf certificate for whatever hostname/SNI was requested, on the fly, signed by a root CA already installed and trusted on the device (the `mitmproxy` /Burp technique). Reason 2 found no evidence of certificate pinning on this traffic, so this would likely pass ordinary validation — *likely*, not confirmed.

Superseded by options 3B and 4 above for the reason already stated: this approach (and 1, 2) intercepts the whole shared domain with no way to scope narrower, and Microsoft's own guidance is explicit that selective interception inside `*.cloud.microsoft` is unsupported and close to the exact shape of thing that causes the Reason 2 failures below. Recorded here for completeness, not as the recommended path.

Whichever option closes Reason 1, it only ever closes Reason 1. Landing the connection on the listener doesn't touch the SignalR relay problem underneath it — Design A/B below would still be the only two shapes available for what to actually do with the traffic once it arrives.

Once captured: forward everything, no per-request filtering — but keep it configurable

A related question worked through in the same discussion: once a process is captured (via 3B or 4), should the listener inspect each request/URL and only forward the ones that look LLM-related to Optimus, passing everything else through directly? **Decided against, for now** — forward everything that process sends to Optimus unconditionally, and defer the scale/cost question that comes with tunneling non-LLM traffic too. Simpler, and it means no new per-request classification logic is needed on top of the per-process scoping 3B/4 already provide.

That's a default, not a hard removal of the capability — the classification logic (URL/path matching to decide "this looks like Copilot chat traffic") is worth building as a **configurable toggle**, off by default, rather than leaving no way to turn it on later if the volume/cost tradeoff changes. This also keeps Design A's rejected approach (a real path this doc already argues against — see above) clearly separate from a legitimate, narrower version of the same idea: filtering *which requests get forwarded*, not *which backend they're forwarded to*.

Reason 2: the traffic is a persistent SignalR WebSocket, not request/response HTTP — and Microsoft documents interception breaking it

Every interception mechanism this codebase has actually built — including the `forward()` /Optimus tunnel every `BaseUrl` agent uses — is shaped around discrete HTTP request/response pairs: a request comes in, gets relayed or answered, a response goes back. Microsoft 365 Copilot's traffic is not that shape, and this is now confirmed at the protocol level, not just at the network-connectivity level.

Official Microsoft documentation confirms the connectivity requirement. Per the admin doc's **WebSockets (WSS) protocol requirements** section:

Verify that your network supports full WSS connectivity from user devices running Microsoft 365 applications to the following domains: Microsoft Copilot enterprise experiences:

`*.office.com` , `*.cloud.microsoft` and `copilot.cloud.microsoft` .

And, listed as a typical cause of "*Copilot application failures*":

Network devices attempting to perform Transport Layer Security (TLS) inspection of connections

Independent reverse-engineering confirms the actual protocol, not just that some WebSocket is involved. A third-party project (`cramt/m365-copilot-proxy`) built a working proxy for the built-in BizChat/Office-web Copilot experience specifically (Word/Excel/etc., not consumer Bing Copilot — the two are architecturally distinct and easy to conflate; an earlier pass of this research briefly did), which required capturing and matching the live wire protocol, not just guessing at it:

- **Transport:** SignalR-over-WebSocket, at `wss://substrate.office.com/m365Copilot/Chathub/{oid}@{tid}` — the auth token rides in the URL query string, not a header.
- **Handshake:** a JSON protocol-negotiation frame (`{"protocol":"json","version":1}`), then typed frames (invocation, stream item, completion, ping, close) separated by `0x1E` record-separator bytes — genuine, verifiable SignalR wire-protocol detail.
- **Keep-alive:** the client must answer server ping frames or the connection dies — a real, bidirectional, stateful protocol, not a one-way stream.

A related doc on Copilot's newer "Cowork" agentic feature ([Cowork network endpoints](#)) is direct about the failure mode a naive interceptor hits:

SSL/TLS inspection isn't supported and might interrupt long-lived SSE connections, which can result in message delivery failures, retries, or frozen tasks.

What this does and doesn't prove. Microsoft's language describes what happens with *generic corporate TLS-inspecting proxy appliances* — hardware typically tuned for short-lived HTTP request/response, which often mishandle long-lived, stateful connections (aggressive idle timeouts, buffering that breaks keep-alives, no awareness of SignalR's own ping/pong contract). It is not a claim that no relay could ever handle this traffic — a purpose-built proxy that understands the SignalR framing, holds the connection open, and answers pings correctly is a solved class of problem in general (this is how API gateways and service meshes handle WebSocket/SignalR traffic routinely). No evidence of application-level certificate pinning (an absolute technical block, distinct from a reliability complaint) was found in this pass — the language throughout is about connection fragility under naive inspection, not a stated refusal to accept a foreign certificate.

A genuinely open, unresolved wrinkle: SignalR's standard client behavior is to `negotiate` over plain HTTP first, receive a list of transports the server supports, and *fall back* to Server-Sent-Events or long-polling when a WebSocket upgrade fails — this exists specifically for networks that block WS upgrades, which is exactly this scenario. The reverse-engineering above didn't observe or document that fallback in practice, so it's unconfirmed whether Microsoft's Chathub deployment actually honors a downgrade or is configured WebSocket-only. If it does downgrade, the interception question changes shape entirely: forcing the WS upgrade to fail might push the client onto a transport (HTTP long-polling) that existing request/response relay infrastructure already knows how to handle, without needing to build a new SignalR-aware WebSocket relay at all. Unconfirmed either way — a real next step if this is ever revisited, not something to treat as settled.

What remains genuinely unverified: whether this codebase's own relay infrastructure (`forward()` /Optimus, or a hypothetical new listener kind) is shaped to proxy a persistent SignalR WebSocket (or, per the wrinkle above, a forced-fallback long-polling transport) without the failure modes Microsoft describes. Everything built so far is request/response-shaped. Either path would

be new, real engineering work, not a reuse of anything that exists today — and it would only be worth attempting after Reason 1 is somehow solved, which it currently isn't.

Update, 2026-09-22 — confirmed directly against Optimus's own source: this capability is real, live, and more capable than described

An internal design discussion first surfaced this: the person who owns Optimus stated *"we have the ability in Optimus to handle that [WSS forwarding]... we basically are supporting that, and if there's a bug, we'll fix it."* That claim has now been checked directly against Optimus's actual codebase (`/Users/umeshk/work/zax/optimus`) — **confirmed, and the real capability goes well beyond bidirectional byte relay:**

- **gateway-wss** (a dedicated crate, own `DESIGN.md`) implements full RFC 6455 frame parsing (`frame.rs`), upgrade-handshake detection (`detect.rs`), and a bidirectional relay (`relay.rs` , `run_wss_relay`) — wired into the live request path in `gateway-proxy/src/pipeline.rs` : `wss_enabled: true` is the actual default (line 308), and a real request is dispatched through `is_ws_upgrade_request` (line 1226) and `run_wss_relay` (line 1769), not a proposal sitting behind a disabled flag.
- **It doesn't just relay — it inspects.** `relay.rs` reassembles fragmented frames into complete UTF-8 text messages and hands them to a pluggable `WsMessageInspector` — the same content-inspection hook (`ws_inspect.rs`) used for policy/DLP scanning elsewhere in Optimus. SignalR's JSON + `0x1E` -framed messages ride on ordinary WebSocket **text** frames, so this reassembly-and-inspect path applies to the Chathub traffic transparently, with no SignalR-specific code needed.
- **A real enforcement mode already exists:** `wss.enforce` (tracked as ZSAI-8160) holds a message's frames until its policy verdict lands, and substitutes a blocked message with a policy-notice frame — masked correctly for whichever direction requires it. This is strictly more than "forward it through" — it's the same block/substitute capability Optimus applies to ordinary HTTP traffic, now extended to a live WebSocket stream.
- **Fail-open discipline is explicit and tested:** an oversized message, a `permessage-deflate` -compressed frame (RSV1, never inflated), or a saturated inspector all forward the original bytes verbatim and are separately counted (`messages_skipped_oversize` , `messages_skipped_compressed` , `messages_skipped_saturated`) rather than silently blocking or hanging the connection — real test coverage exists for each path.

This retires the "pending independent confirmation" caveat this section carried previously.

Reason 2's cost is not "new, cross-team engineering work in a system outside this repo" — it's confirmed, shipped, enabled by default. The triple-JWT-attached-once-at- `Upgrade` detail from the original discussion is consistent with this design (the relay authenticates the stream at the handshake, not per frame). The Design A/B analysis below is kept as a record of the reasoning that applied *before* this was checked, and because the underlying architecture question it raised

(bridging WSS↔HTTP conceptually) is now moot for Optimus specifically — Optimus speaks WSS natively, no bridging required.

Considered: bridge WSS↔HTTP around Optimus's HTTP-only relay (superseded — kept for the record)

Superseded by the confirmed finding above: Optimus's premise here was wrong. This section was written on the assumption that `forward()` — the HTTP request/response tunnel every `BaseUrl` agent uses — was the *only* thing Optimus could do, which would have meant either bridging WSS into that shape or asking for new capability. Direct code inspection found a third, correct answer: Optimus already has a **separate, native WSS relay path** (`gateway-wss`, `run_wss_relay`) alongside `forward()`, so no bridging is needed at all for Optimus specifically. Kept below as a record of the reasoning that applied before this was checked — the analysis is still relevant if this codebase's *own* listener ever needs to speak to a WSS-only backend Optimus doesn't front, just not for this case.

A natural next idea, raised and worked through in this investigation: since Optimus's `forward()` only ever speaks HTTP, could our own listener terminate the client's WebSocket, translate each frame into something Optimus *can* relay, and translate the response back into WS frames on the way out? Worked through fully, this splits into two concrete designs, and both run into a real, structural problem rather than a shallow one.

Design A — bridge to a different, genuinely HTTP-shaped backend. Microsoft does expose an official HTTP/SSE surface for Copilot chat: the Graph `chatOverStream` API, a developer-facing REST endpoint that returns a plain `text/event-stream` response. In principle, the listener could translate each client "send message" frame into a POST against that endpoint, relay it through Optimus unchanged, and translate the SSE response back into WS frames.

The problem: **that endpoint is a different product surface**, built for third-party developers writing their own bots against Copilot — not the backend the built-in Word/Excel UI actually talks to. It almost certainly lacks the same document-grounding, citations, and session context the native experience depends on. This wouldn't be *intercepting* the real exchange; it would be silently substituting a different, likely-degraded conversation for it. That defeats the actual point of wanting this visibility in the first place — governing what really happened, not a lookalike generated on the side.

Design B — a true transport bridge that preserves the real exchange. Encode the actual SignalR frames as opaque bytes in an HTTP body, relay that blob through Optimus untouched (Optimus doesn't need to understand SignalR at all here, just move bytes — within what `forward()` already does), and translate back on the way out.

The problem, as first stated: **something on the far end still has to open a real WebSocket to `substrate.office.com`**. Optimus's `forward()` speaks HTTP to the upstream; it has no "open a

WebSocket to the real backend and tunnel these bytes through it" capability today. This design doesn't eliminate the WebSocket-handling problem — it relocates *who* has to solve it, from this codebase's listener to Optimus itself.

That relocation is a real cost, not a dead end. Optimus isn't code in this repo, but it isn't a true third-party black box either — it's Zscaler's own cloud policy gateway. "Optimus needs to learn to carry a SignalR session" is a legitimate cross-team engineering proposal, not something to write off as out of reach. What it actually requires: Optimus would need to do for a live, stateful WebSocket what `forward()` does today for a single HTTP request/response — hold the connection open for its full duration, preserve (or at least not corrupt) the SignalR frame boundaries, answer keep-alive pings on whichever leg needs them, and still apply policy to the content flowing through it rather than just moving bytes blindly. That's a materially different capability than tunneling one-shot request/response pairs — bigger and longer-lived than anything else in this investigation — but it's an engineering investment to scope and propose, not a structural impossibility.

The sharper, cheaper alternative, and why it's a tradeoff rather than a free win. If instead the listener itself opened the real WebSocket to Microsoft directly — bypassing Optimus entirely — that's mechanically buildable today, with no new dependency on Optimus at all. But that only works safely for traffic that's low-stakes and not worth policy-inspecting. M365 Copilot's chat content is the opposite: it's exactly the traffic worth inspecting — the actual prompts and completions, the real DLP-relevant material. Bypassing Optimus gets the mechanism working fastest, at the cost of discarding the one property that made building it worthwhile in the first place. It's the fallback if the Optimus investment isn't prioritized, not a substitute for it.

Net, updated 2026-09-22: Design A still trades away fidelity (a different, likely-degraded backend) — that hasn't changed. Design B's cost picture has: Optimus's own owner states the SignalR-tunnel capability this section calls "a legitimate cross-team engineering proposal" may already exist, pending independent code-level confirmation (see the update under Reason 2 above). If confirmed, Design B stops being a proposal and starts being a matter of correctly wiring an existing capability. Design B's bypass variant remains available regardless, cheaply, at the cost of the policy enforcement this was supposed to provide. All of this still sits behind Reason 1, which nothing in this section touches.

Net assessment

Blocker	Status	Path forward, and its cost
Reason 1 — no redirect lever	No app-level config lever exists; the endpoint is a fixed, wildcarded, tenant-wide domain	Ranked in priority order above: 3B and 4 are the two live candidates, each with its own design doc and checkpoint findings (3B , 4). Options 1/2/DNS-wildcard all

Blocker	Status	Path forward, and its cost
		share the same disqualifying blast-radius problem and are superseded by 3B/4.
Reason 2 — persistent SignalR WebSocket	Closed, confirmed 2026-09-22 against Optimus's actual source. <code>gateway-wss</code> 's <code>run_wss_relay</code> is live in the default request path (<code>wss_enabled: true</code>), reassembles and inspects text messages generically (covers SignalR transparently), and already has an enforcement mode (<code>wss.enforce</code> , ZSAI-8160) that holds and substitutes blocked messages.	No cost remaining — this is existing, shipped, enabled-by-default capability, not new engineering. Design A/B's bridging analysis is superseded and kept only as a record.

Recommendation: treat this as a resourcing and prioritization decision for Reason 1 only — Reason 2 is closed. As of 2026-09-22, Optimus already does the hard part: native WSS relay, generic text-message reassembly and inspection (covering SignalR's framing without any Copilot-specific code), and a real enforcement mode that can hold and substitute a blocked message. Nothing about routing this traffic through Optimus once it's captured needs new engineering. That leaves Reason 1 as the only real remaining work: confirm whether 3B (per-process proxy, reusing the WebView2 lever) applies to the M365 client, with option 4 (process-aware WFP/Network-Extension redirect) as the buildable fallback if it doesn't — both already have their own design docs. Once Reason 1 closes, this is a straightforward wiring task, not an open research question.

Sources

- [App and network requirements for Microsoft Copilot admins](#) — Microsoft Learn, updated 2026-09-08 (official)
- [Microsoft Copilot Cowork network endpoints \(Preview\)](#) — Microsoft Support (official)
- [Microsoft 365 URLs and IP address ranges](#) — Microsoft Learn (official)
- `cramt/m365-copilot-proxy` — [M365 Copilot API docs](#) — third-party, reverse-engineered against a real working proxy (not Microsoft-published; cited for the specific SignalR/WebSocket protocol detail Microsoft's own docs don't spell out)
- Internal design discussion, 2026-09-22 (`docs/intercept-design-transcript.txt` , `docs/whiteboard-intercept-design.{md,pdf}`) — first-party, including Optimus's own owner on the WSS-handling capability claim; cited for the ranked network-layer alternatives, and as the lead that prompted the Reason 2 code check below

- Optimus source (`optimus` repo, `src/gateway-wss/`, `src/gateway-proxy/src/pipeline.rs`, `src/gateway-proxy/src/ws_inspect.rs`) — direct code inspection, 2026-09-22; confirms Reason 2 closed (see the update under Reason 2)